

TGLib: An Open-Source Library for Temporal Graph Analysis

Lutz Oettershagen
 University of Bonn
 Bonn, Germany
 lutz.oettershagen@cs.uni-bonn.de

Petra Mutzel
 University of Bonn
 Bonn, Germany
 petra.mutzel@cs.uni-bonn.de

Abstract—We initiate an open-source library for the efficient analysis of temporal graphs. We consider one of the standard models of dynamic networks in which each edge has a discrete timestamp and transition time. Recently there has been a massive interest in analyzing such temporal graphs. Common computational data mining and analysis tasks include the computation of temporal distances, centrality measures, and network statistics like topological overlap, burstiness, or temporal diameter. To fulfill the increasing demand for efficient and easy-to-use implementations of temporal graph algorithms, we introduce the open-source library TGLIB, which integrates efficient data structures and algorithms for temporal graph analysis. TGLIB is highly efficient and versatile, providing simple and convenient C++ and Python interfaces, targeting computer scientists, practitioners, students, and the (temporal) network research community.

Index Terms—temporal graph, data mining, centrality, open-source library

I. INTRODUCTION

TGLIB is an open-source C++ template library with an easy-to-use Python front-end focusing on efficient temporal graph analysis tasks. Network data often originates from dynamic systems that change over time: Links are formed or broken, such that the topology of the network changes over time. *Temporal graphs* capture these changes. A temporal graph is a graph that changes over time, i.e., each edge has a time stamp that determines when the edge exists in the graph. Hence, the topology of the graph changes in discrete time steps. Temporal graphs are often good models for real-life scenarios due to the inherently dynamic nature of most real-world activities and processes. In many situations, events, e.g., communication in social networks, are time-stamped, such that temporal graphs naturally arise from the recorded data.

A finite temporal graph consists of a finite set of (static) vertices and a finite set of temporal edges. A temporal edge connects two vertices at a discrete *availability time*, and edge traversal costs a (usually) non-negative amount of time (called the *transition time*). The availability time denotes the time when an edge is available for transition, and the transition time defines how long the transition takes.

This work introduces TGLIB, an open-source toolkit for handling and analyzing temporal graphs. Our library is geared towards a variety of research communities that often need to work with increasingly larger temporal graphs. The applications of temporal graphs are manifold:

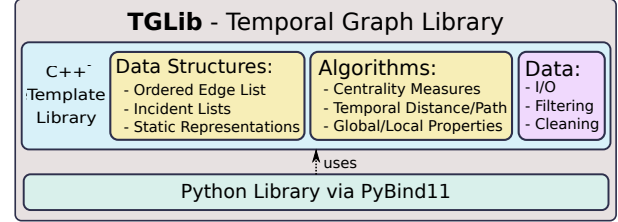


Fig. 1. Overview of the high-level architecture of the TGLIB library. The data structures and temporal graph algorithms are implemented in efficient C++. A Python front-end provides easy-to-use access.

—*Communication networks* are a prime example of the application of temporal graphs. Email (or text message) networks model the (almost) instantaneous communication between the participants, and have been used to identify different dynamics of communication as well as properties of the participants [1]–[3]. Vertices of the network represent the participants and temporal edges represent each communication.

—*Proximity and contact networks* record the contacts between individuals by measuring their proximity. For example, modern smartphones are ubiquitous and can record the proximity of users to identify contacts or build opportunistic networks [4], [5]. Several works discuss the spreading of diseases in contact networks, e.g., [6], [7].

—*Social networks*, formal or informal, are a fundamental part of human life. Nowadays, online social networks, like *Facebook* or *WeChat*, host billions of users. In online and offline social networks, participants join and leave the network over time and form or end relations with each other. Recent works discuss the importance of temporal properties, e.g., [8], [9].

There are many more prominent use-cases for temporal graphs, like modeling transportation networks [10], [11] or applications in biology, e.g., modeling dynamic protein-protein interactions [12], [13], and neural brain networks [14].

Contributions: We introduce TGLIB, an open-source temporal graph library under the permissive MIT license. Our library focuses on temporal distance and centrality computations and other local and global temporal graph statistics. TGLIB is designed for performance and usability by an efficient and modular C++ implementation of the core data structures and algorithms and an easy-to-use Python front-end allowing users and researchers without in-depth (C++) programming

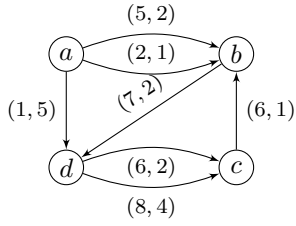


Fig. 2. Example of a temporal graph $\mathcal{G}$. At each edge the availability and transition time is given as pair (t, λ) .

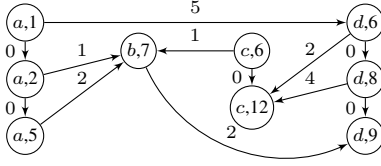


Fig. 3. Example for the static time-respecting representation introduced in [15] of the temporal graphs shown in Figure 2.

experience to use our new library.

Additionally, we offer the first implementations of new algorithms for the earliest arrival distance using the temporal graph data structure introduced in [15], a variant of the fastest path algorithm from [16] for shortest temporal paths, a top- k harmonic closeness algorithms for shortest temporal path distance based on the algorithmic ideas of the top- k closeness algorithm using minimum duration distance [16], and finally, an algorithm for the temporal edge betweenness based on the directed line graph representation [17].

II. PRELIMINARIES

A *temporal graph* $\mathcal{G} = (V, \mathcal{E})$ consists of a finite set of vertices V and a finite set of directed *temporal edges* $\mathcal{E}$. A temporal edge $e = (u, v, t, \lambda)$ consists of the vertices $u, v \in V$, *availability time* (or *time stamp*) $t \in \mathbb{N}$ and *transition time* $\lambda \in \mathbb{N}$, i.e., $e = (u, v, t, \lambda) \in V \times V \times \mathbb{N}^2$. We model an undirected temporal graph by a directed temporal graph using a forward- and a backward-directed edge with equal time stamps and traversal times for each undirected edge.

We use $n = |V|$ and $m = |\mathcal{E}|$ to denote the numbers of vertices and temporal edges, respectively. The *arrival time* of an edge $e = (u, v, t, \lambda)$ (at vertex v) is $t + \lambda$. We use $V(\mathcal{G})$ to denote the set of vertices of $\mathcal{G}$, the minimal number of incoming (outgoing) temporal edges over all vertices by δ_{min}^- (δ_{min}^+). Similarly, we define δ_{max}^- (δ_{max}^+) for the maximal incoming (outgoing) degree. Furthermore, we denote with $N(u) = \{v \mid (u, v, t, \lambda) \in \mathcal{E}\}$ the neighborhood of u . With $T(\mathcal{G})$, we denote the set of all availability times of edges in $\mathcal{G}$, i.e., $T(\mathcal{G}) = \{t \mid (u, v, t, \lambda) \in \mathcal{E}\}$. Given a temporal graph $\mathcal{G} = (V, \mathcal{E})$, it is common to restrict algorithms and computations on $\mathcal{G}$ to a given *restrictive time interval* $I = [a, b]$ with $a, b \in \mathbb{N}$, such that only the temporal subgraph $\mathcal{G}' = (V, \mathcal{E}')$ with $\mathcal{E}' = \{(u, v, t, \lambda) \in \mathcal{E} \mid t \geq a \text{ and } t + \lambda \leq b\}$ needs to be considered.

III. DESIGN GOALS, ARCHITECTURE, AND TEMPORAL GRAPH DATA STRUCTURES

In the following, we will discuss the design goals and motivations for the high-level architecture of TGLIB. Furthermore, we introduce the data structures used for efficiently representing temporal graphs.

A. General Architecture

Figure 1 shows a high-level view of the general architecture of TGLIB. It consists of two main components: 1) the C++ template library and 2) the Python binding. The C++ library provides the generic temporal graph data structures and contains the implementations of temporal graph algorithms. Furthermore, the C++ library offers functionality for IO, filtering, and data cleaning temporal graphs, which are common task necessary for real-world temporal graph data sets.

The Python interface allows non-C++-experts to use the efficient algorithms provided by TGLIB. We used Pybind11¹ for generating the Python binding.

B. Design Goals

We developed TGLIB under the following design goals:

1) *Performance and Efficiency*: All running-time critical code is written in a modern C++ template library focusing on efficiency. Furthermore, many of the algorithms benefit from straightforward shared-memory parallelization; hence, when possible, we use OpenMP for loop parallelization and take advantage of modern shared-memory parallel processing capabilities. The lean temporal graph data structures are templates based and designed for running time and memory efficiency.

2) *Usability and Integration*: The C++ template library is provided as a platform-independent header-only library that can be easily integrated into existing or new C++ projects. Furthermore, we provide an easy-to-use Python interface with clear workflows for temporal graph algorithms and analysis to make TGLIB widely available to researchers and practitioners.

3) *Exentability, Reuseability, and Sustainability*: The C++ source code of TGLIB is based on a generic object-oriented modular designed to be easily extendable and reusable. To this end, we use class templates for the temporal graph data structures to allow extension with customized data. For example, weighted temporal graphs or temporal graphs with time-dependent node labels can be realized by providing corresponding edge or node datatypes. Our implementation is fully documented, and we use unit tests and static code verification to ensure correctness and sustainability [18].

C. Temporal Graph Data Structures

We implemented the temporal graph data structures listed in Table I. The temporal graph data structures are provided as generic class templates. Table I shows the worst-case sizes of the data structures using the implemented default classes. In the following, we discuss the implemented data structures.

¹ <https://github.com/pybind/pybind11>

TABLE I
OVERVIEW OF THE TEMPORAL GRAPH DATA STRUCTURES.

Name	Edge Type	Size	Reference
Temporal Edge Stream (STREAM)	Temporal	$\mathcal{O}(m)$	[19]
Edge Incidence Lists (ILISTS)	Temporal	$\mathcal{O}(n + m)$	[16]
Time-Respecting Static Graph (TRS)	Static	$\mathcal{O}(n + m)$	[15]
Directed Line Graph (DLG)	Static	$\mathcal{O}(m^2)$	[17]
Aggregated Graph (AGGR)	Static	$\mathcal{O}(n^2)$	[20]

TABLE II
OVERVIEW OF THE IMPLEMENTED ALGORITHMS.

Type	Algorithm	Data Structure	Reference
<i>Distance</i>	Earliest Arrival/ Latest Departure	STREAM ILISTS TRS	[19] [21], [22] New
	Min. Transition Sum/ Min. Hops	STREAM ILISTS TRS	[19] New [15]
	Fastest	STREAM ILISTS TRS	[19] [16], [23] [15]
	(In/Out-)Degree	STREAM ILISTS	[14] [14]
	Temporal Closeness	STREAM/ILISTS/TRS	[24]
<i>Centrality</i>	Top-k Closeness (Min. duration)	ILISTS	[16], [23]
	Top-k Closeness (Shortest)	ILISTS	New
	Temporal Edge Betweenness	DLG	New
	Temporal Katz	STREAM	[25]
	Temporal PageRank	STREAM	[26]
	Temporal Walk Centrality	STREAM	[27]
	Burstiness (Edges)	STREAM/ILISTS	[28]
<i>Global/Local Properties</i>	Burstiness (Nodes)	STREAM/ILISTS	[28]
	Temporal Clustering Coeff.	STREAM/ILISTS	[29]
	Temporal Diameter	STREAM/ILISTS/TRS	[20]
	Temporal Efficiency	STREAM/ILISTS/TRS	[29]
	Topological Overlap	STREAM/ILISTS	[30], [31]

1) *Temporal Edge Streams* (STREAM): The temporal graph is given as a sequence of its m edges, chronologically ordered by the availability time of the edges in increasing order, with ties being broken arbitrarily [19]. This representation is often natural when events represented by the edges are sequentially recorded over time. Algorithms for temporal edge streams usually pass over the edges in forward or backward sequential order and are often very efficient. However, the STREAM data structure can be disadvantageous for local computations, e.g., if we are interested in the immediate neighborhood of a single vertex, as we cannot directly access these neighbors.

2) *Edge Incidence Lists* (ILISTS): Here, the temporal graph consists of a set of temporal vertices, and each vertex has a list of temporal edges to its neighbors [16]. The advantage of this representation is the local access to neighbors of a node, which is not directly possible in the temporal edge stream representation.

3) *Static Expansions*: Various static representations of temporal graphs offer different trade-offs between the size of the resulting static graph and the loss of temporal information. We implemented the following versions:

a) *Time-Respecting Static Graph* (TRS): This representation was introduced in [15] and is an improved version

of a data structure from [19]. Here, the temporal graph is transformed into a static, i.e., non-temporal, graph. The time-respecting static graph representation $S(\mathcal{G}) = G_s = (V_s, E_s)$ of a temporal graph $\mathcal{G} = (V, \mathcal{E})$ is defined as follows. First, let $V_o(u) = \{(u, t) \mid (u, v, t, \lambda) \in \mathcal{E}\}$, and $t_m(w) = \max\{t + \lambda \mid (v, w, t, \lambda)\}$ if w has at least one incoming temporal edge. We define $V'(u) = V_o(u) \cup \{(u, t_m(u))\}$ (or $V'(u) = V_o(u)$ if u does not have an incoming edge) and $V_s = \bigcup_{u \in V} V'(u)$. For each temporal edge $(u, v, t, \lambda) \in \mathcal{E}$, we introduce a with λ weighted static edge $((u, t), (v, t'))$ where t' is the smallest arrival time at v larger or equal to $t + \lambda$. Furthermore, for each $u \in V$, the vertices in $V'(u)$ are connected with zero weighted edges in ascending order. Figure 3 shows the TRS of the temporal graph shown in Figure 2.

b) *Directed Line Graph* (DLG): The directed line graph expansion has been previously used for survivability and reliability analysis [32], [33]. In [7], [17], the authors used the DLG for lifting static graph kernels to the temporal domain. In a recent work [27], the authors use the DLG for algebraic weighted walk counting. Given a temporal graph $\mathcal{G} = (V, \mathcal{E})$, the *directed line graph* $DL(\mathcal{G}) = (V', E')$ is the directed graph, where every temporal edge (u, v, t, λ) in $\mathcal{E}$ is represented by a vertex n_{uv}^t , and there is an edge from n_{uv}^t to n_{xy}^s if $v = x$ and $t + \lambda \leq s$.

c) *Aggregated Static Graph* (AGGR): Given a temporal graph $\mathcal{G}$, removing all time stamps and traversal times, and merging resulting multi-edges, we obtain the *aggregated*, or *underlying static*, graph $A(\mathcal{G}) = (V, E_s)$ with $E_s = \{(u, v) \mid (u, v, t, \lambda) \in \mathcal{E}\}$. The edges can be weighted depending on the number of temporal edges, e.g., using the contact frequency, i.e., $\phi((u, v)) = |\{(u, v, t, \lambda) \in \mathcal{E}\}|$. The aggregated graph can be much smaller than the temporal graph as its number of edges is in $\mathcal{O}(n^2)$. However, it does not preserve the temporal information of the network.

IV. IMPLEMENTED ALGORITHMS

Table II gives an overview of the implemented algorithms and the underlying data structures. All implemented algorithms can be restricted to only consider a given time interval I without increasing the running times. In the following, we discuss the implemented algorithms.

A. Temporal Paths, Reachability, and Distances

Finding temporal paths, deciding reachability, and determining temporal distances are essential tasks in various applications and scenarios, e.g., in the computation of temporal centrality measures [16], [34], solving time-dependent transportation problems [35]–[37], or in the simulation and analysis of epidemics [38], [39].

Definition 1. A temporal walk in a temporal graph $\mathcal{G}$ is an alternating sequence $(v_1, e_1, \dots, e_k, v_{k+1})$ of vertices and temporal edges connecting consecutive vertices where for $1 \leq i < k$, $e_i = (v_i, v_{i+1}, t_i, \lambda_i) \in \mathcal{E}$, and $e_{i+1} = (v_{i+1}, v_{i+2}, t_{i+1}, \lambda_{i+1}) \in \mathcal{E}$ the time $t_i + \lambda_i \leq t_{i+1}$ holds. A temporal path P is a temporal walk in which each vertex is visited at most once.

For notational convenience, we omit vertices. The length of a temporal walk ω is the number of edges it contains, and we denote it with $|\omega|$. Let $\omega = (e_1, \dots, e_\ell)$ be a temporal walk in a temporal graph $\mathcal{G}$. The *starting time* of ω is $s(\omega) = t_1$, the *arrival time* is $a(\omega) = t_\ell + \lambda_\ell$, and the *duration* is $d(\omega) = a(\omega) - s(\omega)$. Finally, we define $l(\omega) = \sum_{i=1}^\ell \lambda_i$.

For example, in Figure 2, there are three paths between vertices a and d . The first one consists of only the edge $P_1 = ((a, d, 1, 5))$ and with $d(P_1) = 5$. The second is $P_2 = ((a, b, 2, 1), (b, d, 7, 2))$ with $d(P_2) = 7$. And, path three $P_3 = ((a, b, 5, 2), (b, d, 7, 2))$ with $d(P_3) = 4$.

There are several optimality criteria for temporal paths used in the literature, and we distinguish the following.

Definition 2. Let $\mathcal{G}$ be a temporal graph and $\mathcal{P}$ be the set of all temporal paths in $\mathcal{G}$. A (s, z) -path² $P \in \mathcal{P}$ is

- an earliest arrival path if there is no other (s, z) -path $P' \in \mathcal{P}$ with $a(P') < a(P)$,
- a latest departure path if there is no other (s, z) -path $P' \in \mathcal{P}$ with $s(P') > s(P)$,
- a minimum duration, or fastest, path if there is no other (s, z) -path $P' \in \mathcal{P}$ with $d(P') < d(P)$,
- a shortest path if there is no other (s, z) -path $P' \in \mathcal{P}$ with $l(P') < l(P)$, and
- a minimum hops path if there is no other (s, z) -path $P' \in \mathcal{P}$ with $|P'| < |P|$.

For the example in Figure 2, P_3 is the only fastest (a, d) -path. Notice that the subpath $P'_3 = ((a, b, 5, 2))$ is not a fastest (a, b) -path. The only fastest (a, b) -path consists of edge $(a, b, 2, 1)$ and has a duration of one.

We provide algorithms for determining the temporal distances of Definition 2 for the different data structures, see Table II. The reason for providing the algorithms for different data structures is that depending on the topology of the graph, different algorithms can be more efficient than others, see, e.g., [15], [16], [19]. We additionally introduce a new shortest paths algorithm for the ILISTS data structure and a new earliest arrival algorithm for the TRS data structure³. For each temporal distance, we provide algorithms to obtain an optimal temporal path and the temporal diameter, which is defined as the maximum optimal temporal distance between any two (reachable) vertices in the network [40].

B. Centrality Measures

The centrality of a node (edge) in a network quantifies its structural importance. Various functions can be used to measure node (edge) centrality by assigning values corresponding to some measurement of importance to each node (edge), where the informative value must be assessed based on a research question. For introductions of centrality approaches, see, e.g., [41]–[44]. TGLIB provides the following centrality measures designed explicitly for temporal graphs.

² We use z instead of t as the target vertex because we use t to denote a time stamp. ³ A formal description of the algorithms with correctness and complexity proofs will be in an extended version of this paper.

1) *Temporal Closeness*: Due to the differences in reachability and optimality in temporal graphs, several versions of temporal closeness have been suggested, see, e.g., [16], [45], [46]. Using the optimal distance computations for earliest arrival paths, fastest paths, etc., we provide four different versions of harmonic temporal closeness defined as

$$C(u) = \sum_{v \neq u \in V} \frac{1}{d(u, v)},$$

where we define $1/\infty = 0$ for non-reachable vertices. Furthermore, we provide the top- k approach introduced in [16] for finding the k highest closeness centrality values and the corresponding vertices. The authors of [16] only introduced their top- k algorithm for harmonic closeness wrt. to the minimum duration distance. We provide a new additional implementation for the minimum transition times distance.

2) *Temporal Edge Betweenness*: Similar to the static edge betweenness [47], the temporal edge betweenness is an edge centrality measure and quantifies the importance of the temporal edges in terms of the shortest temporal paths crossing the temporal edge. It can be computed by counting the shortest paths in the directed line graph representation due to the one-to-one mapping of walks in $\mathcal{G}$ and $DL(\mathcal{G})$ [17].

3) *Temporal Katz Centrality*: The *Katz centrality* introduced in [48] measures vertex importance in terms of the number of random walks starting (or arriving) at a vertex, down-weighted by their length. The authors of [25], [49] adapt the walk-based Katz centrality to temporal graphs.

4) *Temporal PageRank Centrality*: Rozenshtein and Gionis [26] incorporate the temporal character in the definition of the static PageRank originally introduced by [50]. They obtain a temporal PageRank by replacing walks with temporal walks.

5) *Temporal Walk Centrality*: Temporal Walk Centrality is a recently proposed centrality measure that aims to rank the vertices according to their ability to obtain and distribute information [27].

C. Further Local and Global Properties

Moreover, we implemented the following local and global temporal graph properties.

1) *Burstiness*: Burstiness measures how much a sequence of contacts τ (of a single node or between a pair of nodes) deviates from the memoryless random Poisson process [20]. It is defined as

$$B(\tau) = \frac{\sigma_\tau - m_\tau}{\sigma_\tau + m_\tau} \in [-1, 1],$$

where σ_τ and m_τ denote the standard deviation and mean of the inter-contact times τ , respectively [28]. A value close to one indicates a very *bursty* sequence, and a value close to minus one a more periodic sequence.

2) *Temporal Clustering Coefficient*: The temporal clustering coefficient is defined as

$$C_C(u) = \frac{\sum_{t \in T(\mathcal{G})} \pi_t(u)}{|T(\mathcal{G})| \binom{|N(u)|}{2}},$$

where $\pi_t(u) = |\{(v, w, t, \lambda) \in \mathcal{E} \mid v, w \in N(u)\}|$ [29].

3) *Temporal Efficiency*: The temporal efficiency is a global statistic based on the temporal closeness values of the nodes [29]. It is defined as

$$T_{\text{eff}}(\mathcal{G}) = \frac{1}{n(n-1)} \sum_{u \in V(\mathcal{G})} \sum_{v \neq u \in V(\mathcal{G})} \frac{1}{d(u, v)}$$

with $d(u, v)$ being a temporal distance and $1/\infty = 0$ in case of non-reachable vertices.

4) *Topological Overlap*: The topological overlap of a node is defined as

$$T_{\text{to}}(u) = \frac{1}{T(\mathcal{G})} \sum_{t=1}^{T(\mathcal{G})} \frac{\sum_{v \in N(u)} \phi_{uv}^t \phi_{uv}^{t+1}}{\sqrt{\sum_{v \in N(u)} \phi_{uv}^t \sum_{v \in N(u)} \phi_{uv}^{t+1}}},$$

where $\phi_{uv}^t = 1$ iff. there exists a temporal edges between u and v at time t and zero otherwise [30], [31]. In case that the denominator equals one, we define $T_{\text{to}}(u) = 1$. And the global topological overlap is defined as $T_{\text{to}}(\mathcal{G}) = \frac{1}{n} \sum_{u \in V} T_{\text{to}}(u)$. The topological overlap lies in the range between zero and one. A value close to zero means many edges change between consecutive time steps, and a value close to one means there are often only a few changes.

V. COMPARISON TO RELATED SOFTWARE

There are several popular graph libraries designed for conventional static graphs, e.g., Networkit [51], OGDF [52], LEMON [53], or Boost graph [54]. However, they are not designed to handle the peculiarities of temporal graphs, e.g., they do not support algorithms that respect the temporal restrictions in temporal walks and paths. The SNAP library provides various algorithms for temporal graphs, like counting specific temporal motifs [55]. We are not aware of a dedicated C++ library for temporal graphs. For Python, the Teneto library is a dedicated temporal graph library supporting various analytical methods [14]. However, the library focuses on analyzing small networks obtained from fMRI brain scans and does not support transition times on the edges. Moreover, it does not perform well on mid-size to large temporal graphs as its mainly based on Python code and matrix-based computations. Finally, Teneto is published under the, compared to the MIT license, more restrictive GNU GPL3 license.

VI. PYTHON INTERFACE

TGLib is available as a precompiled Python package on the Python Package Index (PyPI).⁴ This enables straightforward installation and usage within Python environments. The library can be installed via `pip` as follows:

```
1 pip install temporalgraphlib
```

After installation, the functionality of TGLib is accessible through the `pytglib` module. For example:

```
1 import pytglib as tgl
2
3 # Load a temporal graph from an ordered edge list
4 tgs = tgl.load_ordered_edge_list("example.tg")
5
```

⁴ <https://pypi.org/project/temporalgraphlib/>

TABLE III
OVERVIEW OF THE TEMPORAL GRAPH STATISTICS.

Data set	$ V $	$ \mathcal{E} $	$ T(\mathcal{G}) $	$ E_s $	$\delta_{\max}^-$	$\delta_{\max}^+$
<i>AskUbuntu</i>	159 316	964 437	960 866	596 933	4 926	8 729
<i>Enron</i>	87 101	1 147 126	220 312	321 288	6 165	32 613

```
6 # Compute and print basic statistics
7 stats = tgl.get_statistics(tgs)
8 print(stats)
```

Listing 1. Example Python Usage

This interface provides efficient access to the core temporal graph algorithms and is particularly useful for rapid prototyping and exploratory analysis.

VII. EXAMPLE USE-CASE

For this example use-case, we load two real-world data sets and compare different variants of temporal closeness. The first one is the *AskUbuntu*, a network consisting of interactions on the stack exchange website *Ask Ubuntu* [55]. The second data set is the *Enron* email network between employees of a company [56]. To obtain the basic statistics, we load the temporal graph and call the `get_statistics` function (see Listing 2). Table III shows (a subset) of the returned statistics. Note that Teneto is unable to load these data sets due to its temporal graph representation as a sequence of adjacency matrices and the resulting out-of-memory error (6.13 EiB for *Enron*). Next, we compute the closeness centrality with respect to minimum duration distance and earliest arrival time. We compare the obtained rankings using Kendall's τ rank correlation using SciPy⁵. We obtain correlations of 0.79 for *AskUbuntu* and 0.94 for *Enron*, showing that the two types of temporal closeness are strongly correlated in both graphs.

```
1 import pytglib as tgl
2 import scipy.stats as ss # for correlation
3 tgs = tgl.load_ordered_edge_list("datasetname")
4 stats = tgl.get_statistics(tgs)
5 print(stats)
6 closeness_fastest = tgl.temporal_closeness(tgs,
7     ↪ tgl.Distance_Type.Fastest)
8 closeness_ea = tgl.temporal_closeness(tgs, tgl.
9     ↪ Distance_Type.Earliest_Arrival)
10 tau, p_value = ss.kendalltau(closeness_fastest,
11     ↪ closeness_ea)
```

Listing 2. Example Use-case

VIII. OPEN-SOURCE DEVELOPMENT AND LICENSE

TGLIB free software licensed under the permissive MIT License. It is available at <https://gitlab.com/tgpublic/tglib>. We aim to encourage a diverse community, including network researchers, data mining practitioners, and algorithm engineers, to use TGLIB and contribute to the open-source development.

⁵ <https://scipy.org/>

IX. CONCLUSION AND FUTURE WORK

We introduced the open-source toolkit TGLIB, a C++ library for efficient temporal graph analysis featuring an easy-to-use and accessible Python front-end. So far, we have implemented a wide range of algorithms for distance, centrality, and analytical computations based on various efficient temporal graph data structures. TGLIB offers researchers, practitioners, and students convenient access to temporal graph algorithms. Furthermore, it offers a unified and accessible approach for reproducibility and comparability. We are actively working to integrate further and future methods and algorithms into our library. Using the permissive MIT license, we hope that TGLIB will be used and extended by the temporal graph community.

REFERENCES

- [1] J. Candia, M. C. González, P. Wang, T. Schoenharl, G. Madey, and A.-L. Barabási, “Uncovering individual and collective human dynamics from mobile phone records,” *Journal of physics A: mathematical and theoretical*, vol. 41, no. 22, p. 224015, 2008.
- [2] J.-P. Eckmann, E. Moses, and D. Sergi, “Entropy of dialogues creates coherent structures in e-mail traffic,” *Proceedings of the National Academy of Sciences*, vol. 101, no. 40, pp. 14 333–14 337, 2004.
- [3] P. Holme, C. R. Edling, and F. Liljeros, “Structure and time evolution of an internet dating community,” *Social Networks*, vol. 26, no. 2, pp. 155–174, 2004.
- [4] C. Avin, M. Koucký, and Z. Lotker, “How to explore a fast-changing world (cover time of a simple random walk on evolving graphs),” in *International Colloquium on Automata, Languages, and Programming*. Springer, 2008, pp. 121–132.
- [5] A. Chaintreau, A. Mibaa, L. Massoulié, and C. Diot, “The diameter of opportunistic mobile networks,” in *Proceedings of the 2007 ACM Conference on Emerging Network Experiment and Technology, CoNEXT*, J. Kurose and H. Schulzrinne, Eds. ACM, 2007, p. 12.
- [6] M. Ciaperoni, E. Galimberti, F. Bonchi, C. Cattuto, F. Gullo, and A. Barrat, “Relevance of temporal cores for epidemic spread in temporal networks,” *Scientific reports*, vol. 10, no. 1, pp. 1–15, 2020.
- [7] L. Oettershagen, N. M. Kriege, C. Morris, and P. Mutzel, “Classifying dissemination processes in temporal graphs,” *Big Data*, vol. 8, no. 5, pp. 363–378, 2020.
- [8] S. Hanneke and E. P. Xing, “Discrete temporal models of social networks,” in *ICML Workshop on Statistical Network Analysis*. Springer, 2006, pp. 115–125.
- [9] A. Moinet, M. Starnini, and R. Pastor-Satorras, “Burstiness and aging in social temporal networks,” *Physical review letters*, vol. 114, no. 10, p. 108701, 2015.
- [10] R. Gallotti and M. Barthélemy, “The multilayer temporal network of public transport in great britain,” *Sci. data*, vol. 2, no. 1, pp. 1–8, 2015.
- [11] N. Huynh and J. Barthélemy, “A comparative study of topological analysis and temporal network analysis of a public transport system,” *International Journal of Transportation Science and Technology*, 2021.
- [12] S. Lebre, J. Becq, F. Devaux, M. P. Stumpf, and G. Lelandaïs, “Statistical inference of the time-varying structure of gene-regulation networks,” *BMC systems biology*, vol. 4, no. 1, pp. 1–16, 2010.
- [13] T. M. Przytycka, M. Singh, and D. K. Slonim, “Toward the dynamic interactome: it’s about time,” *Briefings in bioinformatics*, vol. 11, no. 1, pp. 15–29, 2010.
- [14] W. H. Thompson, P. Brantefors, and P. Fransson, “From static to temporal network theory: Applications to functional brain connectivity,” *Network Neuroscience*, vol. 1, no. 2, pp. 69–99, 2017.
- [15] S. Gheibi, T. Banerjee, S. Ranka, and S. Sahni, “An effective data structure for contact sequence temporal graphs,” in *2021 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 2021, pp. 1–8.
- [16] L. Oettershagen and P. Mutzel, “Efficient top-k temporal closeness calculation in temporal networks,” in *IEEE International Conference on Data Mining (ICDM)*. IEEE, 2020, pp. 402–411.
- [17] L. Oettershagen, N. M. Kriege, C. Morris, and P. Mutzel, “Temporal graph kernels for classifying dissemination processes,” in *SIAM International Conference on Data Mining (SDM)*. SIAM, 2020, pp. 496–504.
- [18] S. Roth, *Clean C++: Sustainable Software Development Patterns and Best Practices with C++ 17*. Apress, 2017.
- [19] H. Wu, J. Cheng, S. Huang, Y. Ke, Y. Lu, and Y. Xu, “Path problems in temporal graphs,” *Proc. VLDB End.*, vol. 7, no. 9, pp. 721–732, 2014.
- [20] P. Holme and J. Saramäki, “Temporal networks,” *Physics reports*, vol. 519, no. 3, pp. 97–125, 2012.
- [21] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [22] B. Bui-Xuan, A. Ferreira, and A. Jarry, “Computing shortest, fastest, and foremost journeys in dynamic networks,” *International Journal of Foundations of Computer Science*, vol. 14, no. 02, pp. 267–285, 2003.
- [23] L. Oettershagen and P. Mutzel, “Computing top-k temporal closeness in temporal networks,” *KAIS*, pp. 1–29, 2022.
- [24] N. Santoro, W. Quattrociocchi, P. Flocchini, A. Casteigts, and F. Amblard, “Time-varying graphs and social network analysis: Temporal indicators and metrics,” *arXiv preprint arXiv:1102.0629*, 2011.
- [25] F. Béres, R. Pálovics, A. Oláh, and A. A. Benczúr, “Temporal walk based centrality metric for graph streams,” *Applied Network Science*, vol. 3, no. 1, pp. 32:1–32:26, 2018. [Online]. Available: <https://doi.org/10.1007/s41109-018-0080-5>
- [26] P. Rozenshtein and A. Gionis, “Temporal pagerank,” in *ECML PKDD*, ser. LNCS, vol. 9852. Springer, 2016, pp. 674–689.
- [27] L. Oettershagen, P. Mutzel, and N. M. Kriege, “Temporal walk centrality: Ranking nodes in evolving networks,” in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 1640–1650.
- [28] K.-I. Goh and A.-L. Barabási, “Burstiness and memory in complex systems,” *EPL (Europhysics Letters)*, vol. 81, no. 4, p. 48002, 2008.
- [29] J. Tang, M. Musolesi, C. Mascolo, and V. Latora, “Temporal distance metrics for social network analysis,” in *Proceedings of the 2nd ACM workshop on Online social networks*, 2009, pp. 31–36.
- [30] A. Clauset and N. Eagle, “Persistence and periodicity in a dynamic proximity network,” *arXiv preprint arXiv:1211.7343*, 2012.
- [31] J. Tang, S. Scellato, M. Musolesi, C. Mascolo, and V. Latora, “Small-world behavior in time-varying graphs,” *Physical Review E*, vol. 81, no. 5, p. 055101, 2010.
- [32] G. Khanna, S. K. Chaturvedi, and S. Soh, “Two-terminal reliability analysis for time-evolving and predictable delay-tolerant networks,” *Recent Advances in Electrical & Electronic Engineering*, vol. 13, no. 2, pp. 236–250, 2020.
- [33] Q. Liang and E. Modiano, “Survivability in time-varying networks,” *IEEE Trans. on Mobile Computing*, vol. 16, no. 9, pp. 2668–2681, 2016.
- [34] D. Braha and Y. Bar-Yam, “Time-dependent complex networks: Dynamic centrality, dynamic motifs, and cycles of social interactions,” in *Adaptive Networks*. Springer, 2009, pp. 39–50.
- [35] M. Gendreau, G. Ghiani, and E. Guerriero, “Time-dependent routing problems: A review,” *Computers & operations research*, vol. 64, pp. 189–197, 2015.
- [36] A. Idrı, M. Oukarfi, A. Boulmakoul, K. Zeitouni, and A. Masri, “A new time-dependent shortest path algorithm for multimodal transportation network,” *Procedia Computer Science*, vol. 109, pp. 692–697, 2017.
- [37] E. Pyrga, F. Schulz, D. Wagner, and C. Zaroliagis, “Efficient models for timetable information in public transportation systems,” *Journal of Experimental Algorithmics (JEA)*, vol. 12, pp. 1–39, 2008.
- [38] J. Enright and R. R. Kao, “Epidemics on dynamic networks,” *Epidemics*, vol. 24, pp. 88–97, 2018.
- [39] H. H. K. Lentz, A. Koher, P. Hövel, J. Gethmann, C. Sauter-Louis, T. Selhorst, and F. J. Conraths, “Disease spread through animal movements: a static and temporal network analysis of pig trade in germany,” *PloS one*, vol. 11, no. 5, p. e0155196, 2016.
- [40] M. Calamai, P. Crescenzi, and A. Marino, “On computing the diameter of (weighted) link streams,” in *Intl. Symposium on Experimental Algorithms*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2021.
- [41] K. Das, S. Samanta, and M. Pal, “Study on centrality measures in social networks: a survey,” *Social Network Analysis and Mining*, vol. 8, no. 1, p. 13, 2018.
- [42] A. Landherr, B. Friedl, and J. Heidemann, “A critical review of centrality measures in social networks,” *Business & Information Systems Engineering*, vol. 2, no. 6, pp. 371–385, 2010.
- [43] F. A. Rodrigues, “Network centrality: an introduction,” in *A Mathematical Modeling Approach from Nonlinear Dynamics to Complex Systems*. Springer, 2019, pp. 177–196.
- [44] A. Saxena and S. Iyengar, “Centrality measures in complex networks: A survey,” *CoRR*, vol. abs/2011.07190, 2020. [Online]. Available: <https://arxiv.org/abs/2011.07190>

- [45] R. K. Pan and J. Saramäki, "Path lengths, correlations, and centrality in temporal networks," *Physical Review E*, vol. 84, no. 1, p. 016105, 2011.
- [46] P. Crescenzi, C. Magnien, and A. Marino, "Finding top-k nodes for temporal closeness in large temporal graphs," *Algorithms*, vol. 13, no. 9, p. 211, 2020.
- [47] U. Brandes, "On variants of shortest-path betweenness centrality and their generic computation," *Social networks*, vol. 30, no. 2, pp. 136–145, 2008.
- [48] L. Katz, "A new status index derived from sociometric analysis," *Psychometrika*, vol. 18, no. 1, pp. 39–43, 1953.
- [49] P. Grindrod, M. C. Parsons, D. J. Higham, and E. Estrada, "Communicability across evolving networks," *Physical Review E*, vol. 83, no. 4, p. 046120, 2011.
- [50] L. Page, S. Brin, R. Motwani, and T. Winograd, "The pagerank citation ranking: Bringing order to the web." Technical Report 1999-66, November 1999, previous number = SIDL-WP-1999-0120. [Online]. Available: <http://ilpubs.stanford.edu:8090/422/>
- [51] C. L. Staudt, A. Sazonovs, and H. Meyerhenke, "Networkit: A tool suite for large-scale complex network analysis," *Network Science*, vol. 4, no. 4, pp. 508–530, 2016.
- [52] M. Chimani, C. Gutwenger, M. Jünger, G. W. Klau, K. Klein, and P. Mutzel, "The open graph drawing framework (ogdf)," *Handbook of graph drawing and visualization*, vol. 2011, pp. 543–569, 2013.
- [53] B. Dezső, A. Jüttner, and P. Kovács, "Lemon—an open source c++ graph template library," *Electronic Notes in Theoretical Computer Science*, vol. 264, no. 5, pp. 23–45, 2011.
- [54] J. Siek, L.-Q. Lee, A. Lumsdaine *et al.*, *The boost graph library*. Pearson India, 2002, vol. 243.
- [55] A. Paranjape, A. R. Benson, and J. Leskovec, "Motifs in temporal networks," in *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining*, 2017, pp. 601–610.
- [56] B. Klimt and Y. Yang, "The enron corpus: A new dataset for email classification research," in *European Conference on Machine Learning*. Springer, 2004, pp. 217–226.